Lee atentamente este documento y solicita todas las aclaraciones que estimes en la página de la asignatura en la plataforma web

Practica 1 UBU_TPMP (Test Project Management Platform)

Es importante recordar que el objetivo principal de esta práctica es servir de introducción a las pruebas de software. Por este motivo, no es necesario implementar funcionalidades completas ni abordar sistemas complejos; se trata de afianzar los conceptos y familiarizarse con los procesos esenciales relacionados con la validación y verificación de aplicaciones. El enfoque estará en comprender y aplicar los fundamentos, más que en alcanzar un producto final exhaustivo.

Sección C – creación de una aplicación Razor Pages preparada para pruebas web

Objetivo

El objetivo de esta práctica es construir una aplicación web mínima con ASP.NET Core Razor Pages, partiendo de una plantilla generada por Visual Studio e incorporando los ficheros proporcionados por el docente de la asignatura.

La aplicación implementará un proceso básico de identificación en dos pasos y quedará preparada para realizar:

1. Pruebas funcionales manuales.
2. Pruebas web automatizadas con Katalon Studio.
3. Pruebas automatizadas con Selenium WebDriver.
4. Ejecución de las pruebas Selenium sobre Google Chrome.
5. Comprobaciones de navegación, validación, mensajes de error y acceso a páginas protegidas.

La finalidad no es desarrollar una aplicación completa, sino disponer de un entorno web pequeño, controlado y reproducible sobre el que diseñar y ejecutar pruebas. Posteriormente lo mejoraremos.

Proceso y resultados

Diseño

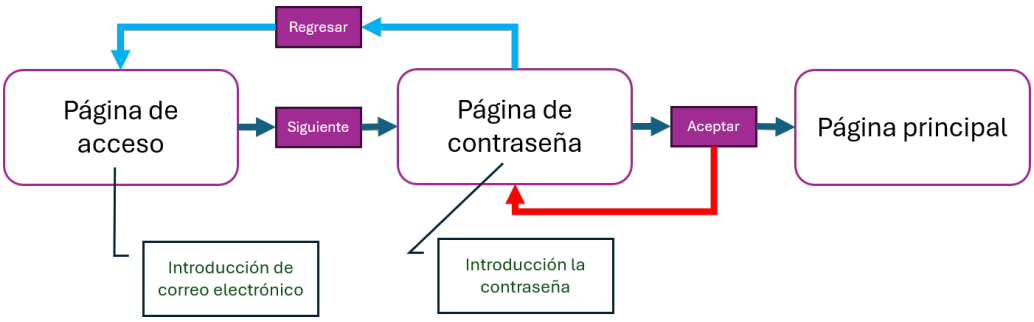


Ilustración 1.- Diagrama transición básico

Requisitos previos

El estudiante deberá disponer de:

1) Clases del núcleo

a) Usuario.cs
b) ResultadoHash.cs
c) ServicioHash.cs
d) Rol.cs
e) Proyecto.cs
f) EntradaLog.cs

2) Capa de datos simulada
a) IDatos.cs
b) Datos.cs

Material proporcionado

Páginas de Login

Se proporcionan los ficheros que contienen:

- Interfaz de introducción del correo.
- Interfaz de introducción de la contraseña.
- Validaciones básicas.
- Navegación entre los dos pasos.
- Uso provisional de Session.
- Identificadores HTML preparados para las pruebas.

Ficheros necesarios:

Index.cshtml
Index.cshtml.cs
Index2.cshtml
Index2.cshtml.cs

Hojas de estilos

Ficheros necesarios:

login.css
login-password.css

Utilidades de la aplicación web

Ficheros necesarios:

ClavesSesion.cs

Se usa para centralizar los nombres para

Login.EmailPendiente
Usuario.Id
ProyectoActual.Id

Creación del proyecto WEB

Proceso

1) Abre el proyecto en Visual Studio y haciendo uso del menú contextual de la solución agrega un nuevo proyecto.
   a) Seleccione la plantilla “Aplicación web de ASP.NET Core (Razor Pages)”
      i) Nombre de la solución: **www**
      ii) Framework: **.NET 10**
      iii) Tipo de autenticación: **Ninguna**
      iv) Configurar para HTTPS: **activado**
      v) Contenedores: **desactivado**
2) Antes de modificar la plantilla:
   a) Compila la solución.
   b) Ejecuta el proyecto.
   c) Comprueba que el navegador se abre.
   d) Comprueba que aparece la página inicial de la plantilla.
   e) Deten la ejecución.
   f) Revisa la ventana Lista de errores.
3) Limpia el proyecto dejando



4) Antes hay que tener en cuenta que estructuralmente que de cada fichero con extensión “cshtml” puede depender un fichero con el mismo nombre extensión “cs” y que no debe ser eliminado. Posteriormente veremos que son dos ficheros que físicamente se almacenan por separado.
5) Edita el fichero Pages\index.cshtml.cs

```
@page
@model www.Pages.IndexModel
@{
}
<p>Redirigiendo...</p>
```

6) Edita el fichero Pages\index.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace www.Pages;

public class IndexModel : PageModel
{
    public IActionResult OnGet()
    {
        return RedirectToPage("/Login/Index");
    }
}
```

7) Entra en el explorador de archivos (puedes hacerlo desde el menú contextual del proyecto), crea las carpetas necesarias y copia desde el fichero www.zip disponible dentro del apartado práctica 1 de la asignatura



8) Reemplaza los ficheros "_layout.cshtml" y "_layout.cshtml.cs" dentro de "www\pages\shared". Contienen la cabecera y el pie

Comprobaciones

Nombre de las páginas Razor

Ten cuidado con las ubicaciones, si lo prefieres puedes cambiar el nombre, pero también debes hacerlo en el menú localizado en la cabecera (dentro del fichero .

Comprobaciones

Espacios de nombres

1) Comprueba en los ficheros el espacio de nombres los espacios de nombres.
   a) Los archivos ".cshtml.cs" deben utilizar un espacio de nombres coherente con la carpeta:
      namespace PracticaPruebasWeb.Web.Pages.Login;
   b) La directiva @model debe apuntar al PageModel correcto:
      @model PracticaPruebasWeb.Web.Pages.Login.IndexModel
2) Comprueba la incorporación de los estilos CSS
   a) Enlazar el CSS
      i) Añada en las páginas del login:
         <link rel="stylesheet" href="~/css/login.css" />1
3) Comprueba las referencias entre proyectos
4) Comprueba el registro de la capa de datos
   a) En Program.cs, registre la implementación proporcionada.
      builder.Services.AddSingleton<IDatos, Datos>();

      La elección de Singleton implica que todos los usuarios de esa ejecución comparten la misma instancia en memoria. En esta práctica se utilizará únicamente como simulación docente.

Actividades

1) Familiarízate con el contenido de los ficheros de una página Razor.
2) Investiga el uso de variables de sesión.
3) Analiza el fichero "Layout".

Preparación de la aplicación para la automatización

Los controles relevantes deben tener identificadores HTML claros y estables.

Comprueba cada uno de ellos en todas las páginas

Los identificadores estables permitirán localizar elementos desde Selenium, por ejemplo:

Comprobaciones previas a la creación de las pruebas

- La solución compila sin errores.
- La aplicación se inicia.
- La página inicial corresponde al login.
- El CSS se carga correctamente.

[1] El carácter ~ representa la raíz web de la aplicación.

Creación de las pruebas

En la entrega se deben incluir:

- las pruebas realizadas con Katalon Recorder.
- Implementación de las pruebas usando las librerías de Selenium, MsTest y el controlador para Chrome

Tips para Selenium

Se añadirá un proyecto de pruebas a la solución y se incorporará Selenium WebDriver. Selenium utiliza WebDriver para controlar navegadores y el inicio de una sesión en C# puede realizarse mediante ChromeDriver.